CH 1 — Intro to Object Oriented Programming

Luke Lu • 2026-11-07

The course is taught in 3 perspectives

1. Programmer's perspective: efficiency, bug face
2. Designer's perspective: basic software engineering, desing patterns
3. Compiler's perspective: what must a compiler to to support a language feature

> 🐳 **Info — C++ History**
>
> Invented in 1980s
>
> C with classes is the earlier name of C++
>
> Currently using C++20 with "modules" introduced

Hello World C++ Program

Program "01-intro/hello.cc"

```
import <iostream>;

using namespace std;

int main(){ // required for C++ to have return type as int for main()
  cout << "Hello World" << endl;
}
```

Command for running C++ programs:

Compile the system header

```
g++20h

g++15.2.0           #compiler using

-std=c++20          #standard

-fmodules-ts        #use module tech specs

-c                  #separately compiling a part of a program

-x c++system-header

iostream            #module name
```

Then compile the rest of the program

```
g++20

g++15.2.0           #compiler using

-std=c++20          #standard

-fmodules-ts        #use module tech specs

-wall               #warn all

-g                  #debug

hello.cc

-o                  # output
```

For this course, `include`, `stdio.h`, `printf` are banned.

Instead `std::cout << value <<std::endl` is used.

We can avoid `std::` by having `using namespace std`

IO Streams

"<<" is output operator, ">>" is input operator

Importing `iostream` gives access to three variables:

`std::cout`	type: `ostream`	`cout << x;` `//print values stoerd in x on stdout`
`std::cerr`	type: `ostream`	`cerr << "Error" << error << endl;` `//print error message to stderr`
`std::cin`	type: `istream`	`cin >> x;` `//reads from stdin to store in x`

Program "01-intro/plus.cc"

```
import <iostream>;

using namespace std;

int main(){
  int x, y;
  cin >> x >> y;
  cout << x+y << endl;
}
```

Info — I/O information

- Reading ints ignores leading white spaces
- After attempting to read, checking for success is important
- If a reads fails, th expression "`cin.fail()`" will be true
  - If a read fails, all the subsequent reads fails. (unless specified)
  - When a read fails, we most acknowledge the failure and to attempt resuming reading using
    - `cin.clear();` to clear the flag
    - `cin.ignore();` to ignore the one that caused the flag to be raised.
- If a read fails due to end of input(eof), the expression "`cin.eof()`" will be true

Implicit/Automatic Conversion:

C++ will automatically treat `cin` as a `boolean` value: `!cin == cin.fail()`

Program "02-io/readInts.cc"

Goal: Read and print all ints from stdin. Stop in case of failure.

```
import <iostream>;
using namespace std;

int main(){
  int x;
  while(true){
    cin >> x; //expression statement, and evaluates to cin
    if(cin.fail()) break; //we can replace "cin.fail()" to "!(cin >> x)"" or "!cin"
    cout << x << endl;
  }
}
```

Program "02-io/readInts5.cc"

Goal: Read and print all ints from stdin. Ignore all non-ints. Stop at eof.

```
import <iostream>;
using namespace std;

int main(){
  int x;
  while(true){
    if(cin >> x){
      cout << x << endl;
    }else if(cin.eof()){
      break;
    }else{
      cin.clear(); //clear the flag raised when reads fails for reading non-ints
      cin.ignore(); //ignore the one that caused the flag to be raised
```

```
    }
  }
}
```

I/O manipulators

```
int x = 95;
cout << hex << x; //print in hexadecimal
cout << dec; //go back to base-10

//boolalpha, setprecision, skipws, noskipws, etc...
```

Strings

Strings

```
import <string>;
//Automatic memory managed

string s = "h"; //Creating a C++ string from a C-style string

string s{"h"}; //Uniform Initialization of strings
```

Note: C++ does not have strings unless importing the module.

Operations	C	C++
Comparison	`strcmp(s1, s2)`	`s1 == s2`
Length	`strlen() //O(n)`	`s1.length()//O(1)`
Random Acess	`str[i]`	`str[i]`
Concatenation	`strcat(s1, s2)`	`s1 + s2`

Note: in C++, operators can be recoded as functions

```
string vars;

cin >> vars; //Here, ">>" is a function call >>(cin, vars);

//Skip leading whitespace and stop at frist whitespace. Reads a sequence of non-
whitespace characters.

getline(cin, s); //From cin to read a line
```

F-stream

We can read from and write to other sources/destination

```
import <fstream>;
```

`ifstream`: reads from a file

`ofstream`: writes to a file

```
import <fstream>;

ifstream myfile{"rooms.txt"}; //opens the file for reading

//myfile behaves like cin
```

```
string s;

myfile >> s; //reads the first word from the file
```

Note:

Everything we can do with `cin`(`iostream`), we can do with `ifstream` variable

Everything we can do with `cout`(`iostream`), we can do with `ofstream` variable

```
import <iostream>;
import <fstream>;
using namespace std;

int main(){
  ifstream myfile{"rooms.txt"};
  string s;
  while(myfile >> s){
    cout << s << endl;
  }
}
```

When `myfile` goes of out of scope, the file is automatically closed.

```
import <sstream>;

istringstream iss; //

ostringstream oss; //

int num1 = 0;
int num2 = 5;

oss << "What assignment between" << num1;
oss << "and" << num2 << "is due next week?" << endl;

strings = oss.str();

//"What assignment between 0 and 5 is due next week?"
```

S-stream

Using an `istringstream` to convert a `string` to an `int`

Example:

Program “02-io/getNum.cc”

```
import <iostream>;
import <sstream>;
using namespace std;

int main(){
  int n;
  while(true){
    cout <<"Enter a number:" << endl;
    string s;
    cin >> s;
    if(istringstream {s} >> n) break; //"istingstream {s}" is unamed variable
```

```
    //if(istringstream iss{s} >> n) break; iss be only in scope of the if statement
  }
  cout << "You entered " << n << endl;
}
```

Program “01-intro/readInts5.cc” vs “01-intro/readIntsSS.cc”

Inputs	readInts5.cc	readIntsSS.cc
`Hello123`	123 as output	no outputs

Arguments

Command Line Argument

```
int main(int argc, char *argc[]){}

// argc is the numbers of arguments to the program
// char *argc[] is each argument as a C-style string
```

Default Arguments

```
void printfile(string name="myfile.txt"){
  ifstream is{name};
  string s;
  while(is >> s){
    cout<<s<<endl;
  }
}

printfile(); //translates to printfile("myfile.txt");
printfile("file.txt"); //takes file.txt as input that replaces myfile.txt
```

Rules:

1. Defualt argument must appear last
   - The last parameter(or last `n` parameters can have default value)

   ```
   void foo(int x = 0, string temp);//ILLEGAL

   void foo(string temp, int x = 0)); //LEGAL
   ```

2. If we are going to rely on default values, we can leave out the last(or lthe last `n`) arguments

   ```
   void foo(int x = 0, string s = "bla");

   foo(5, "bar"); //LEGAL
   foo(5); //LEGAL
   foo(); //LEGAL

   foo("bar"); //ILLEGAL
   foo(, "bar"); //ILLEGAL
   ```

   `foo` acts as 3 different functions:

   ```
   void foo(int x, string s);
   void foo(int x);
   void foo();
   ```

Function Overloading

> 📘 **Definition — Function Overloading**
>
> - Can write function with same name as long as the number or types of parameters or order of parameters makes each function distinct
> - Difference in the return type is not enough

Example:

```
21 >> 3; //right bit shift operator, built-in

int x;
cin >> x; //stream operator, imported
//The above translates to operator>>(cin, x), which calls operator>>(istream, int)
string s;
cin >> s; //Tanslates to operator>>(cin, s), which calls operator>>(istream, string)

s1 + s2; //Translates to operator+(s1, s2), which calls operator+(string, string)
```

Structs

In C++

```
struct Node{
  int data;
  Node *next;
}

Node n={s, nullptr};

//nullptr means null pointers, which is better than NULL
```

Constants

```
const int x= 5; //constants cannot be left uninitialized

const Node n = {s, nullptr};
//n cannot be modifiy, n.data cannot be modified, and n.next cannot be modified

int n=5; int m = 10;
const int *p = &n; //p is a pointer to a constant integer

p = &m; //LEGAL
*p = 10; //ILLEGAL
n = 10; //LEGAL

int const* p = &n; //p is a pointer to a constant integer

p = &m; //LEGAL
*p = 10; //ILLEGAL
n=10; //LEGAL


const int x = 5;
```

```
int *p = &x; //ILLEGAL
const int *p = &x; //LEGAL
```

References

Parameter Passing

Pass by Value	Pass by Pointer

`void inc(int x){`
`  x = x+1;`
`}//Pass by value`
 	
`int y = 10;`
`inc(y);`
 	
`cout<<y; //prints 10`
	`void inc(int *x){`
	`  *x = *x+1;`
	`}//Pass by reference`
	
	`int y = 10;`
	`inc(&y);`
	
	`cout<<y; //prints 11`

We do not need `cin >> &x` instead of `cin >> x` because of references

> 📘 **Definition — L-Values and R-Values**
>
> **L-Values** referes to a location
>
> - All variables are **L-Values**
> - Arrays and expressions of form `arr[i]` are **L-Values**
> - That appears on the left-hand-side of an assignment(informal definition)
>
> **R-Values** anything that are not **L-Values**
>
> - Temporary values or computations(i.e. x+y)
> - `struct`

L-Value Reference

- Acts as an constant pointer with automatic dereference

```
int x = 10;
int &y = x; //y is a l-value reference to x
y = 15; //x becomes 15

int *p = &y; //p points to x
sizeof(y); //gives sizeof(x), y is an alias for x
```

The & in the last example, the first usage means is a l-value reference to somthing.

Just like * can be act as pointer when creation and also means dereferencing.

Rules of References

1. Cannot leave l-value references uninitialized
   - Must be initialized to an l-value
2. Cannot create a pointer to a reference
3. Cannot create a reference to a reference

4. Cannot create an array of references
5. You can pass by reference

Example:

Pass by Value	Pass by Pointer	Pass by Reference

`void inc(int x){`
`  x = x+1;`
`}//Pass by value`
 	
`int y = 10;`
`inc(y);`
 	
`cout<<y; //prints 10`
	`void inc(int *x){`
	`  *x = *x+1;`
	`}//Pass by reference`
	
	`int y = 10;`
	`inc(&y);`
	
	`cout<<y; //prints 11`
		`void inc(int &n){`
		`  n =n+1;`
		`  // n is another name for y`
		`}`
		`int y = 10;`
		`inc(y);`
		
		`cout<<y; //prints 11`

So the answer the signature of the `cin >> x` is

```
std::istream &operator>>(std::istream &in, int &n){}
```

- streams cannot be copied
- integer is passed by reference so that changes to n as visible to caller
- return type is cascadeing to others, stream cannot be copied

```
struct uwstudents{};

uwstudents all{}; //takes a lot of memory

void computeStandings(uwstudents student){}
computeStandings(all);
```

- Pass by value will be expensive
- Passing a pointer to `all` would be efficient

```
void computeStandings_v2(uwstudents &student){}
computeStandings_v2(all);
```

Removing & is not a legal function overload

- Passing a reference would be efficient
- `student` is an alias to `all`

```
void printStudentOnCNDP(const uwstudents &s){}
```

- efficient
- `s` is an alias to `all`
- `s` cannot be mutated

> 💡 **Tip** — Prefer to pass by reference to `const`

```
void f(int &n){}
void g(const int &n){}

f(5); //ILLEGAL
g(y+y); //ILLEGAL
```

```
g(5); //LEGAL
g(y+y); //LEGAL
```

Note: `const` allows to pass by reference with r-values.

Dynamic/Heap Memory

- Anytime allocated must deallocate after use
- For `malloc()`, use `free()`, banned for 246
- For `new`, use `delete`

```
int main(){
  struct Node{
    int data;
    Node *next
  };

  Node *np1 = malloc(sizeof(Node)); //Banned for CS246
  Node *np2 = new Node; //Functionally identical
  Node n;

  int *arr1 = malloc(3*sizeof(int));
  int arr2[3];
  int *arr3 = new int[3];

  free(np1);
  delete np2;
  //n is in stack, no need to free

  free(arr1);
  //arr2 is in stack, no need to free
  delete [] arr3;

}
```

Stack Frame	Heap Memory
`np1`	data
	next
`np2`	data
	next
`n` • data • next	
`arr1`	0
	1
	2
`arr2` • 0 • 1	

• 2	
arr3	0
	1
	2

```
int **p = new int*[5]; //An array of 5 Arrays

for(int i = 0; i < 5; i++){
  p[i] = new int[i+1];
}

for(int i = 0; i< 5; i++){
  delete [] p[i];
}

delete [] p;
```

Suppose

Return By Value	ILLEGAL	ILLEGAL

`Node getNode(){`
`  Node n;`
`  return n;`
`}`
	`Node *getNode(){`
	`  Node n;`
	`  return &n;`
	`}`
		`Node &getNode(){`
		`  Node n;`
		`  return n;`
		`}`

• Return by value • Copies from the callee • Stack frame to the caller	Dangling pointer	Out of scope

Operator Overloading

C++ allows to specify the behaviour of operators for user defined types

```
struct vector{int x; inty;};
vector v1{}, v2{};
vector v3 = add(v1, v2);
vector v3 =  v1+ v2; //C++ allows for implementation
vector v4 = 3 * v1;
```

We can implement operator + as a function

- Arg1: left-hand-side(pass by reference to `const`)
- Arg2: right-hand-side(pass by reference to `const`)
- Return type: vector by value(after function call, it is no longer in stack frame)

```
vector operator+(const vector &v1, const vector &v2){
  return {v1.x+v2.x; v1.y+v2.y};
}
```

```
vector operator*(int s, const vector &v1){
  return {s * v1.x; s*v1.y};
}
```

Note: after these implementations, `vector v5 = v1 * 3` is no longer valid

So we create another * as a function using the previous *

```
vector operator*(const vector &v1, int s){
  return s * v1;
}
```

Overload << and >>

```
struct Grade{int grade;};
Grade g{};
cin >> g;
cout << g <<endl;


ostream &operator<<(ostream &out, const Grade &g){ //need to put down flags when
error
  out << q.grade << "%";
  return out;
}

istream &operator>>(istream &in, Grade &g){
  in >> g.grade;
  if (g.grade < 0) g.grade = 0;
  if (g.grade > 100) g.grade = 100;
  return in;
}
```

It is legal because `ostream` is a global variable

C++ Modules/Separate Compilations

Interface: type definitions, function declarations

Implementation: function definitions

In C and old C++, interface files are `.h` files, implementation files are `.c / .cc` files

C++ Modules

Example:

Interface file: `vector.cc`

```
export module vector; //creates a module

export struct vector{
  int x;
  int y;
```

```
}

export vector &operator+(const vector &, const vector &);
```

Client file: `main.cc`

```
import vector;
import <iostream>; //<> tells compiler the module is a system module

int main(){
  vector v{1, 2};
  vector v2 = v+v;
}
```

Implementation file `vector-impl.cc`, you can have multiple implementation file.

```
module vector; //part of the existing vector module

vector &operator+(const vector &v1, cosnt vector &v2){
  return vector {v1.x+v2.x, v1.y+v2.y};
}

//ILLEGAL to have type definition as in the interface file, resulting to error
```

Compiling

```
g++20h iostream sstream fstream...

g++20 -c vector.cc ... #should be compile first because others dependents on it
g++20 -c vector-impl.cc main.cc ...
#b/c the order does not matter for vector-impl.cc and main.cc, we can put it in one
line, but not together with vector.cc

g++20 main.o vector-impl.o vector.o
```